

```
from google.colab import files

uploaded = files.upload()
```

Choose Files retail_sales_dataset.csv

retail_sales_dataset.csv(text/csv) - 51673 bytes, last modified: 5/30/2026 - 100% done
Saving retail_sales_dataset.csv to retail_sales_dataset.csv

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

import warnings
warnings.filterwarnings('ignore')
```

```
df = pd.read_csv("retail_sales_dataset.csv")
```

```
df.head()
```

	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per Unit	Total Amount	
0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150	
1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000	
2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30	
3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500	
4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100	

Next steps:

Generate code with df

New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Transaction ID         1000 non-null  int64  
1   Date                  1000 non-null  object  
2   Customer ID           1000 non-null  object  
3   Gender                 1000 non-null  object  
4   Age                   1000 non-null  int64  
5   Product Category      1000 non-null  object  
6   Quantity              1000 non-null  int64  
7   Price per Unit        1000 non-null  int64  
8   Total Amount          1000 non-null  int64  
dtypes: int64(5), object(4)
memory usage: 70.4+ KB
```

```
df.shape
```

```
(1000, 9)
```

```
df.columns
```

```
Index(['Transaction ID', 'Date', 'Customer ID', 'Gender', 'Age',
       'Product Category', 'Quantity', 'Price per Unit', 'Total Amount'],
      dtype='object')
```

```
df.describe()
```

1 to 8 of 8 entries Filter ?

index	Transaction ID	Age	Quantity	Price per Unit	Total Amount
count	1000.0	1000.0	1000.0	1000.0	1000.0
mean	500.5	41.392	2.514	179.89	456.0
std	288.8194360957494	13.681429659122532	1.1327343409145405	189.68135627129138	559.997631555123
min	1.0	18.0	1.0	25.0	25.0
25%	250.75	29.0	1.0	30.0	60.0
50%	500.5	42.0	3.0	50.0	135.0
75%	750.25	53.0	4.0	300.0	900.0
max	1000.0	64.0	4.0	500.0	2000.0

Show 25 per page
Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Data Cleaning

```
df.isnull().sum()
```

	0
Transaction ID	0
Date	0
Customer ID	0
Gender	0
Age	0
Product Category	0
Quantity	0
Price per Unit	0
Total Amount	0

dtype: int64

```
df.duplicated().sum()
```

np.int64(0)

```
df.drop_duplicates(inplace=True)
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
#   Column             Non-Null Count  Dtype
---  -
0   Transaction ID      1000 non-null  int64
1   Date                1000 non-null  datetime64[ns]
2   Customer ID         1000 non-null  object
3   Gender              1000 non-null  object
4   Age                 1000 non-null  int64
5   Product Category    1000 non-null  object
6   Quantity            1000 non-null  int64
7   Price per Unit      1000 non-null  int64
8   Total Amount        1000 non-null  int64
dtypes: datetime64[ns](1), int64(5), object(3)
memory usage: 70.4+ KB
```

```
df['Month'] = df['Date'].dt.month
```

```
df['Year'] = df['Date'].dt.year
```

```
df['Day_Name'] = df['Date'].dt.day_name()
```

```
df['Quarter'] = df['Date'].dt.quarter
```

df.head()

	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per Unit	Total Amount	Month	Year	Day_Name	Quarter
0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150	11	2023	Friday	4
1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000	2	2023	Monday	1
2	3	2023-	CUST003	Male	50	Electronics	1	30	30	1	2023	Friday	1

Next steps: [Generate code with df](#) [New interactive sheet](#)

Exploratory Data Analysis

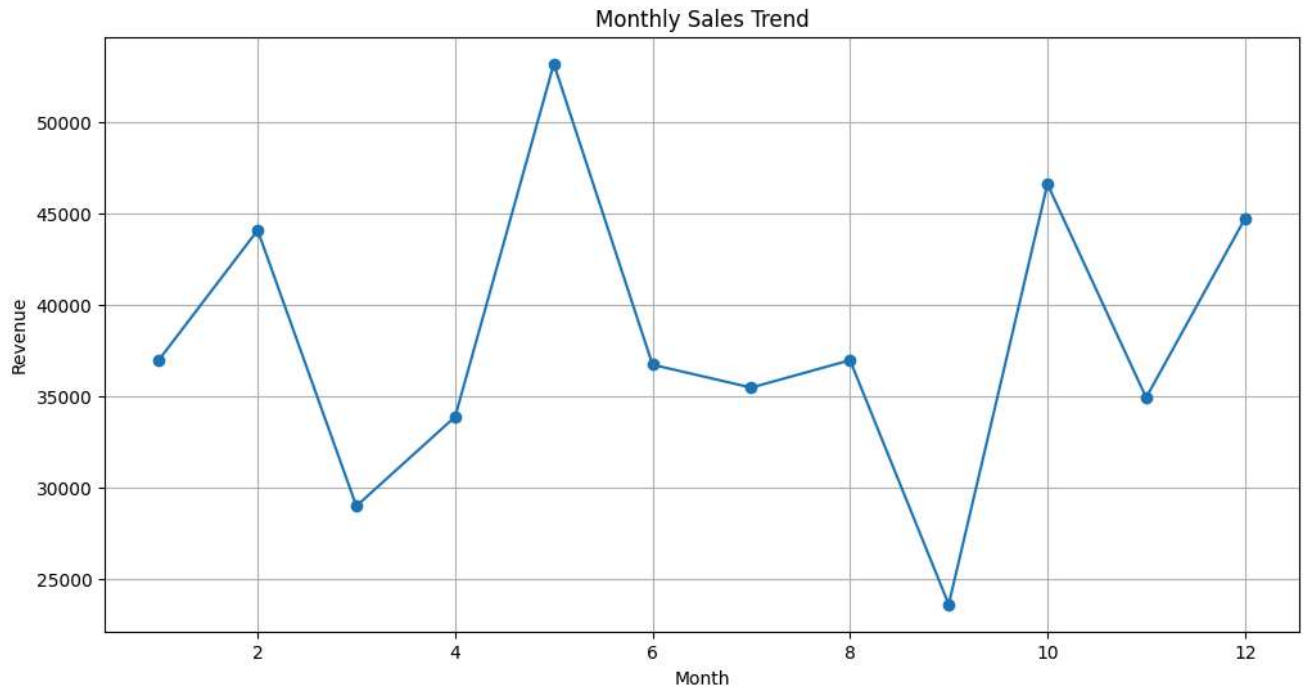
```
monthly_sales = df.groupby('Month')['Total Amount'].sum()

plt.figure(figsize=(12,6))

monthly_sales.plot(
    marker='o'
)

plt.title("Monthly Sales Trend")
plt.ylabel("Revenue")
plt.grid()

plt.show()
```



```
plt.savefig("monthly_sales_trend.png")
```

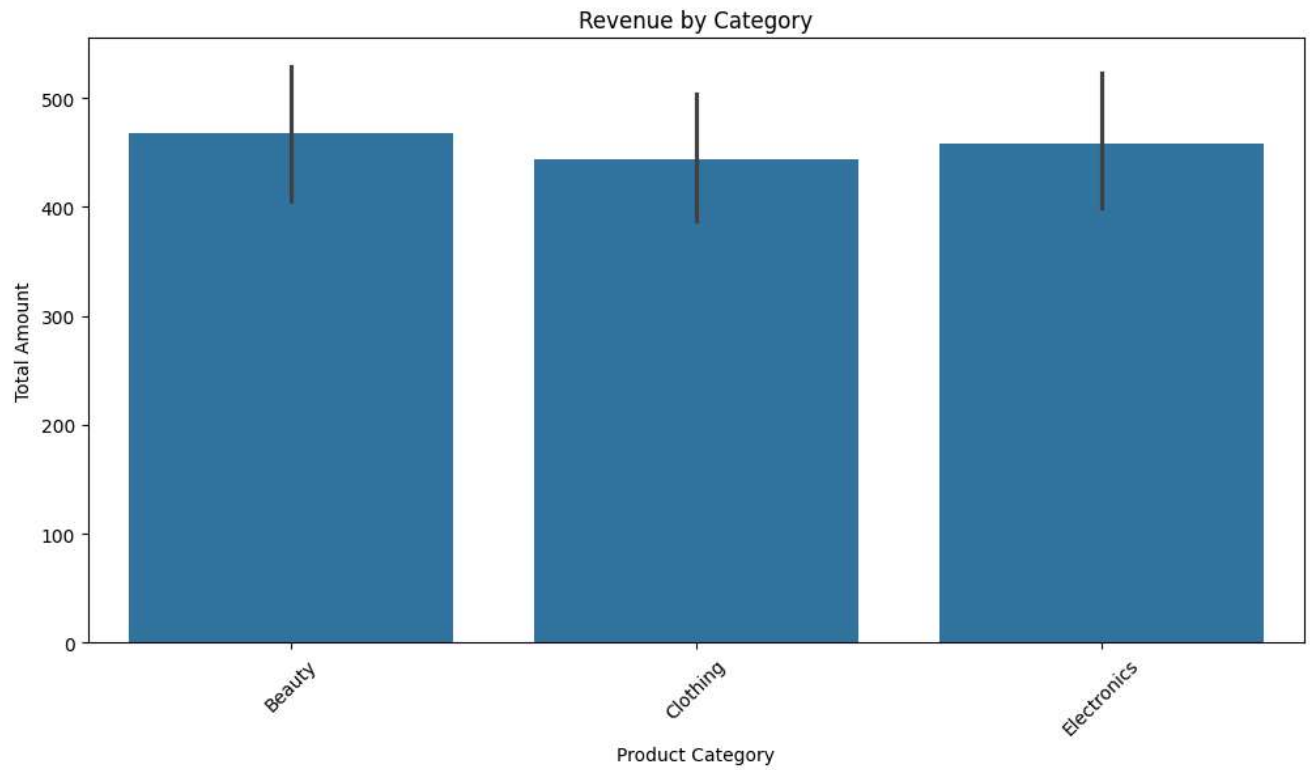
<Figure size 640x480 with 0 Axes>

```
plt.figure(figsize=(12,6))

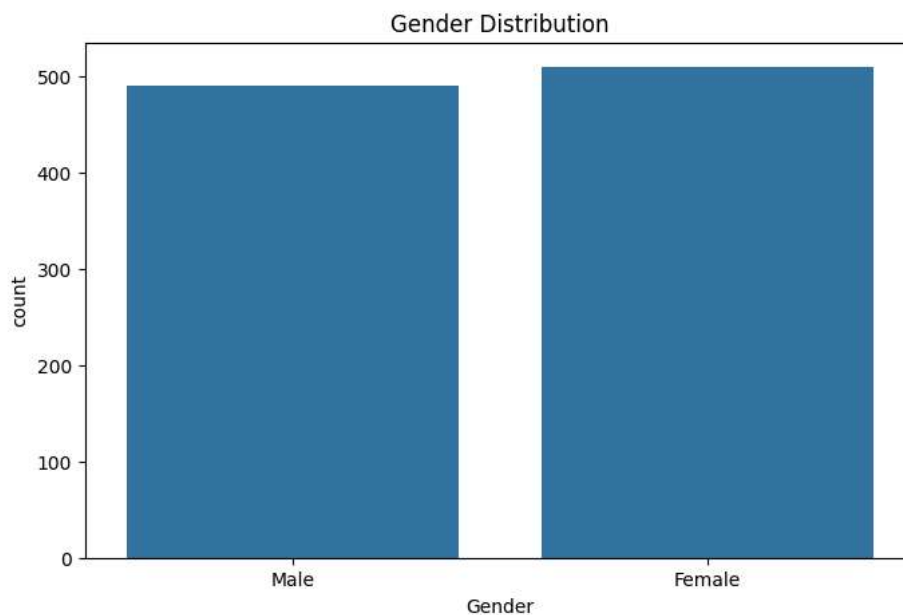
sns.barplot(
    x='Product Category',
    y='Total Amount',
    data=df
)

plt.xticks(rotation=45)
```

```
plt.title("Revenue by Category")  
plt.show()
```



```
plt.figure(figsize=(8,5))  
  
sns.countplot(  
    x='Gender',  
    data=df  
)  
  
plt.title("Gender Distribution")  
plt.show()
```



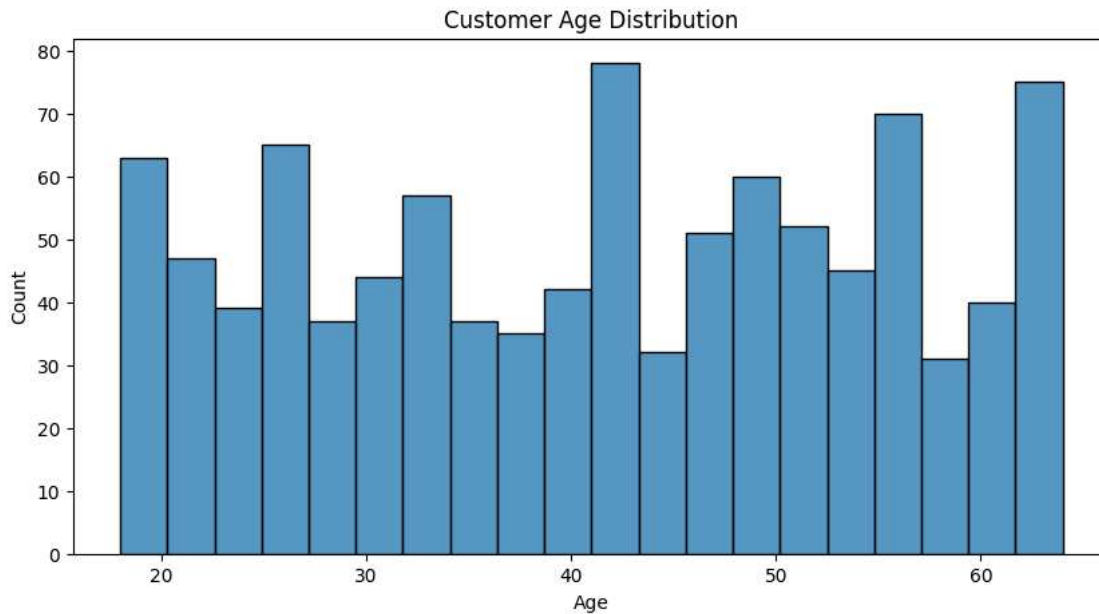
```
plt.figure(figsize=(10,5))  
sns.histplot(  

```

```
df['Age'],
bins=20
)

plt.title("Customer Age Distribution")

plt.show()
```



```
# Customer Segmentation
```

```
customer_spend = df.groupby(
    'Customer ID'
)['Total Amount'].sum().reset_index()
```

```
customer_spend.head()
```

1 to 5 of 5 entries Filter ?

index	Customer ID	Total Amount
0	CUST001	150
1	CUST002	1000
2	CUST003	30
3	CUST004	500
4	CUST005	100

Show 25 per page

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Next steps: [Generate code with customer_spend](#) [New interactive sheet](#)

```
scaler = StandardScaler()

scaled_data = scaler.fit_transform(
    customer_spend[['Total Amount']]
)
```

```
kmeans = KMeans(
    n_clusters=4,
    random_state=42
)

customer_spend['Cluster'] = kmeans.fit_predict(
    scaled_data
)
```

```
plt.figure(figsize=(10,6))
```

```
sns.scatterplot(
```

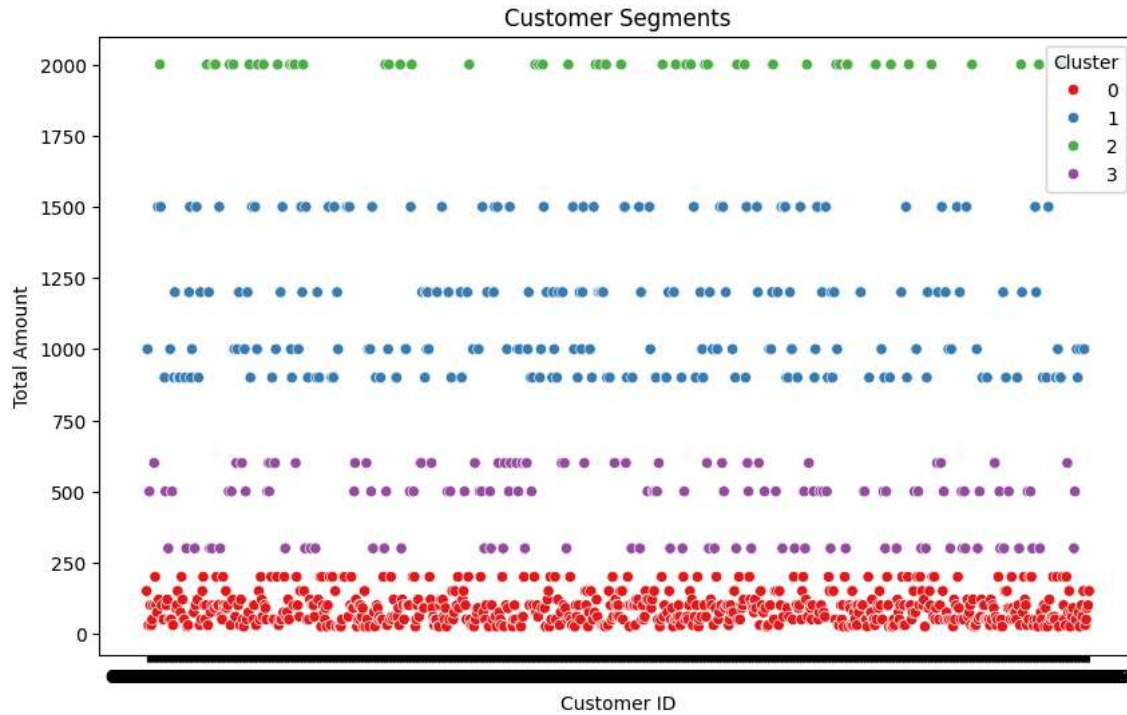
```

x='Customer ID',
y='Total Amount',
hue='Cluster',
data=customer_spend,
palette='Set1'
)

plt.title("Customer Segments")

plt.show()

```



```
# RFM Analysis
```

```
snapshot_date = df['Date'].max()
```

```

rfm = df.groupby('Customer ID').agg({
    'Date': lambda x: (snapshot_date - x.max()).days,
    'Customer ID': 'count',
    'Total Amount': 'sum'
})

```

```

rfm.columns = [
    'Recency',
    'Frequency',
    'Monetary'
]

```

```
rfm.head()
```

1 to 5 of 5 entries Filter ?

Customer ID	Recency	Frequency	Monetary
CUST001	38	1	150
CUST002	308	1	1000
CUST003	353	1	30
CUST004	225	1	500
CUST005	240	1	100

Show 25 per page

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Next steps: [Generate code with rfm](#) [New interactive sheet](#)

```
# Sales Forecasting
```

```
daily_sales = df.groupby(  
    'Date'  
)['Total Amount'].sum()
```

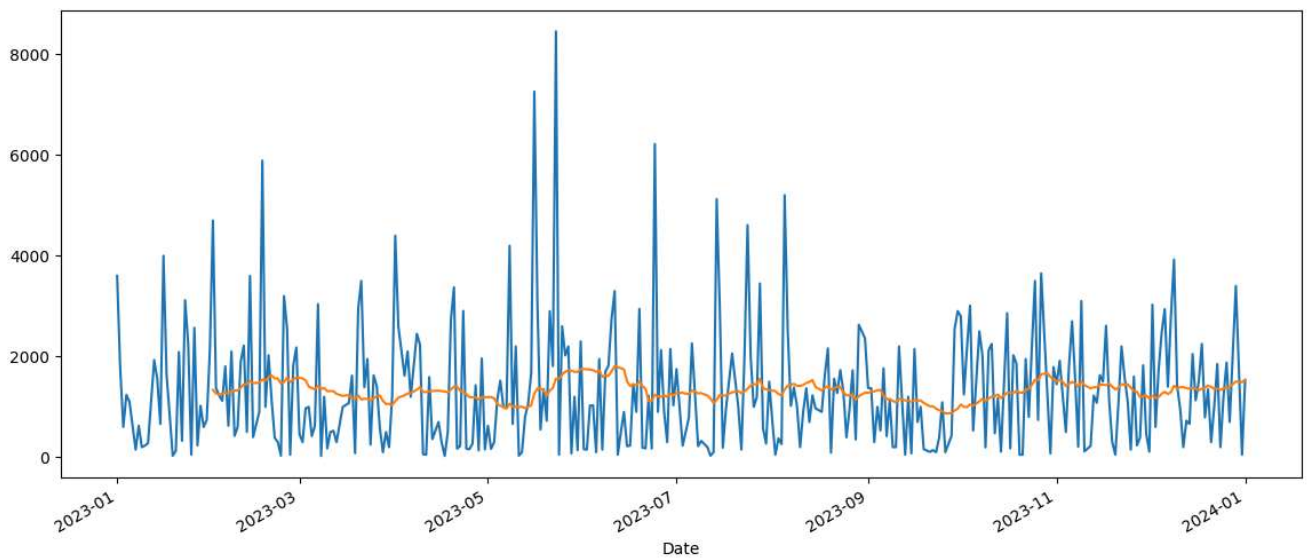
```
daily_sales.head()
```

Date	Total Amount
2023-01-01	3600
2023-01-02	1765
2023-01-03	600
2023-01-04	1240
2023-01-05	1100

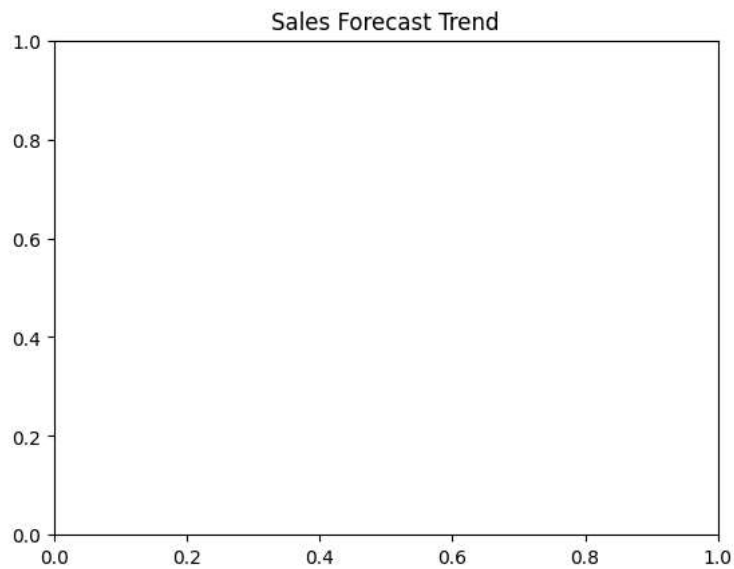
```
dtype: int64
```

```
daily_sales.plot(  
    figsize=(14,6)  
)  
  
daily_sales.rolling(  
    window=30  
) .mean().plot()
```

<Axes: xlabel='Date'>



```
plt.title(  
    "Sales Forecast Trend"  
)  
  
plt.show()
```



```
# Business Recommendations
```

1. Electronics category generates highest revenue.
2. Female customers contribute more purchases.
3. Cluster 2 customers are premium customers.
4. Q4 generates maximum revenue.
5. Future sales show upward trend.

```
plt.savefig(  
    "chart_name.png",  
    bbox_inches='tight'  
)
```

<Figure size 640x480 with 0 Axes>